VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessors – Microcontrollers

Laboratory Report 4

Cooperative Scheduler

Advisor(s): Phan Văn Sỹ

Student(s): Vũ Thiện Nhân – 2452888

HO CHI MINH CITY, NOVEMBER 2026

Table of Contents

1. Introduction 3
2. Hardware Design 3
   2.1 Schematic of the Test Circuit 4
   2.2 Pin Assignment 4
3. Scheduler Design 5
   3.1 Architecture 5
   3.2 Data Structure 6
   3.3 The Delta List 7
   3.4 Operation of the Scheduler 8
4. STM32 Project Configuration 8
5. Scheduler Implementation 10
   5.1 scheduler.h 10
   5.2 scheduler.c 11
6. Application: Tasks and Main Program 16
   6.1 Tasks 16
   6.2 Initialisation and Main Loop 17
7. Simulation Results 18
   7.1 Test Setup 18
   7.2 Virtual Terminal Output 19
   7.3 Analysis of the Timestamps 22
   7.4 Deleting a Task 23
8. Compliance with the Requirements 23
9. Conclusion 24
10. Demo Video and Links 25

1. Introduction

This report presents the implementation of Laboratory 4 (Cooperative Scheduler) of the Microprocessors – Microcontrollers course, using the STM32F103C6 microcontroller simulated in Proteus 8 Professional and programmed with STM32CubeIDE (HAL library).

In the previous laboratories the program was a super loop in which every activity polled a flag or waited for a software timer. This laboratory goes one level up: a cooperative scheduler is built so that functions (tasks) can be called periodically, or once after a delay, with only one hardware timer. The timer interrupt only counts down the time of the tasks; the tasks themselves run to completion in the main loop, one after the other.

The handout (Sections 4 and 5) asks for:

- at least four functions: `SCH_Update()`, `SCH_Dispatch_Tasks()`, `SCH_Add_Task()` (returns the ID of the task) and `SCH_Delete_Task()` (deletes a task by its ID);
- a main program with five tasks running periodically every 0.5 s, 1 s, 1.5 s, 2 s and 2.5 s;
- a demonstration with a regular 10 ms timer tick, a callback registered every 10 ms that prints the time, several timeouts registered at the same time and some calls to `SCH_Add_Task()` before the main loop is entered;
- a design that accepts many timeouts, delivers the callbacks in the right order and avoids O(n) searches in `SCH_Update()`.

The test application of this report toggles five LEDs from five periodic tasks, runs a one-shot task, prints a timestamp every 10 ms and finally deletes the 10 ms task after 3 s. All messages are sent to a Virtual Terminal connected to USART1 (115200 baud) in the form `[time ms] message`. As in the previous laboratories the LEDs are active-low: the anode is tied to 3.3 V and the cathode to a GPIO pin, so driving the pin to logic 0 turns the LED on.

2. Hardware Design

Figure 2.1 shows the Proteus schematic used in this laboratory and Table 2.1 lists the pin assignment.

2.1 Schematic of the Test Circuit

The schematic contains the STM32F103C6 (U1), five red LEDs D1–D5 and a Virtual Terminal. The anode of every LED is connected to +3.3 V and its cathode to a net label (LED_1 … LED_5) on a microcontroller pin. The RXD pin of the Virtual Terminal is

connected to PA9 (USART1_TX) and its TXD pin to PA10 (USART1_RX), so every message sent with `HAL_UART_Transmit()` appears in the terminal window.

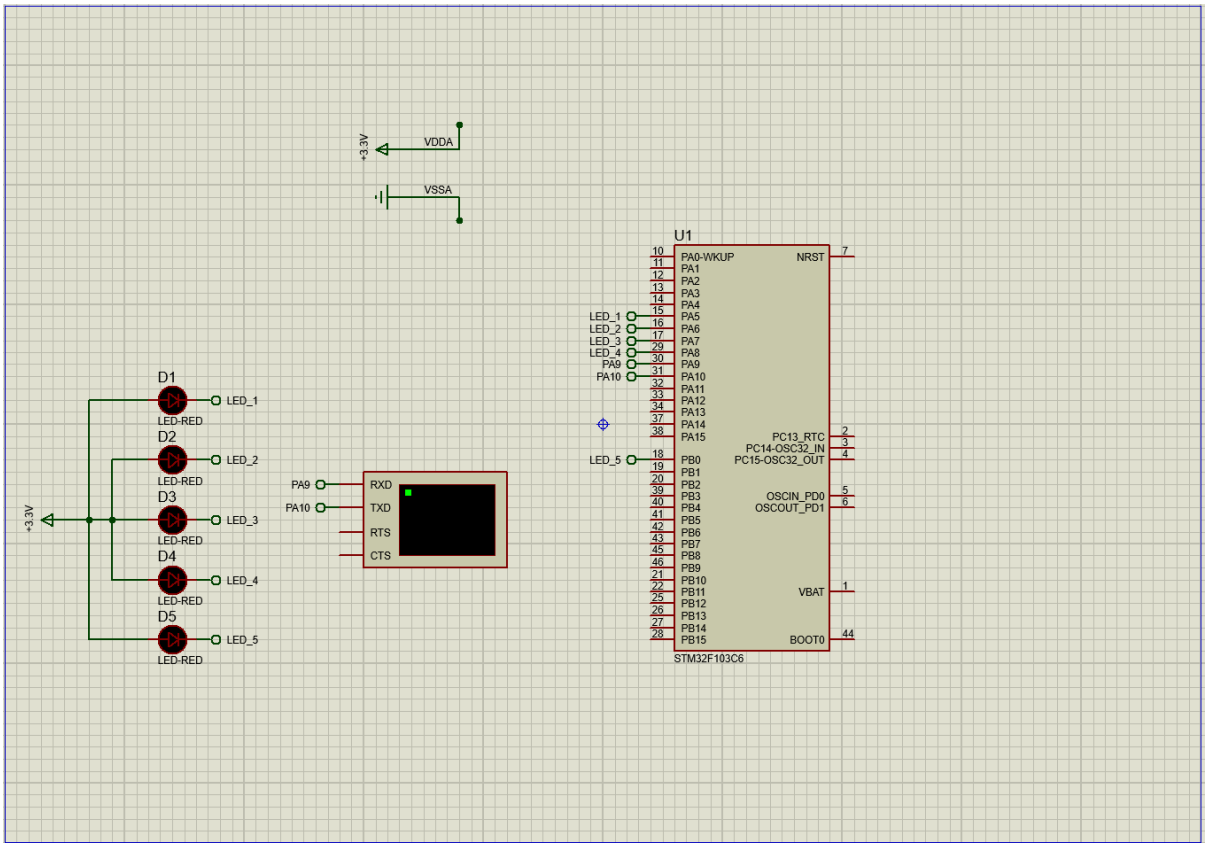


Figure 2.1: Proteus schematic (STM32F103C6, five LEDs and a Virtual Terminal)

2.2 Pin Assignment

Table 2.1 — Pin assignment (LEDs are active-low)

Function	Label	Pin	Configuration
LED 1 (period 0.5 s)	LED_1	PA5	GPIO_Output, lit at 0
LED 2 (period 1 s)	LED_2	PA6	GPIO_Output, lit at 0
LED 3 (period 1.5 s)	LED_3	PA7	GPIO_Output, lit at 0
LED 4 (period 2 s)	LED_4	PA8	GPIO_Output, lit at 0
LED 5 (period 2.5 s)	LED_OS (net label LED_5)	PB0	GPIO_Output, lit at 0
UART transmit	USART1_TX	PA9	Alternate function, to RXD of the Virtual Terminal
UART receive	USART1_RX	PA10	Alternate function, from TXD of the Virtual Terminal (not used by the firmware)

The net label of the fifth LED in Proteus is LED_5, while its CubeMX user label is LED_OS, so the generated macros are `LED_OS_Pin` and `LED_OS_GPIO_Port`. The other four LEDs are on GPIOA and use the macros `LED_1_Pin` … `LED_4_Pin`.

3. Scheduler Design

3.1 Architecture

The scheduler follows the split of the handout between an “update” part, executed in the timer interrupt, and a “dispatch” part, executed in the super loop (Figure 3.1).

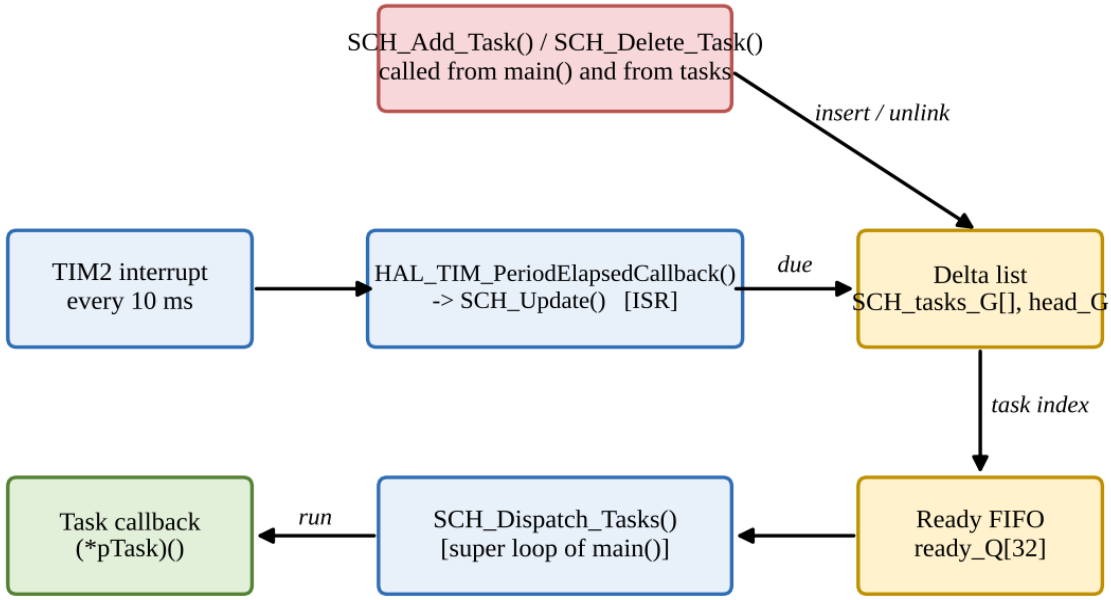


Figure 3.1: Architecture of the scheduler

- **Timer interrupt.** TIM2 interrupts every 10 ms and the HAL callback calls `SCH_Update()`. It increments the tick counter, counts down the delta list and moves the tasks that are due into the ready FIFO.
- **Delta list.** The waiting tasks are stored in the array `SCH_tasks_G[]`, linked in the order in which they become due.
- **Ready FIFO.** The tasks that are due and wait for the CPU are queued in the order in which they became due.
- **Dispatcher.** `SCH_Dispatch_Tasks()` takes the tasks from the FIFO one by one and runs each one to completion.

The split is necessary for reliability (Section 2.3.6 of the handout): if `SCH_Update()` called the tasks directly, the tick interrupt would effectively be blocked while a long task runs and ticks would be missed. Here the interrupt keeps counting ticks while a task executes; a task that runs late is delayed but it is never lost.

Table 3.1 — Functions of the scheduler

Function	Called from	Role
SCH_Init()	main()	Clears the task array, the list, the FIFO, the tick counter and the error code
SCH_Add_Task(pFunction, DELAY, PERIOD)	main(), tasks	Finds a free slot, inserts the task in the delta list and returns its ID (SCH_MAX_TASKS if it fails)

Function	Called from	Role
SCH_Update()	TIM2 interrupt	Counts a tick, counts down the head of the list, moves due tasks to the FIFO and re-inserts periodic tasks
SCH_Dispatch_Tasks()	main loop	Runs the due tasks in FIFO order and frees the slot of a one-shot task after it has run
SCH_Delete_Task(taskID)	main(), tasks	Removes a task from the list and clears its slot; returns SCH_RETURN_NORMAL or SCH_RETURN_ERROR
SCH_Get_Tick(), SCH_Get_Time_Ms()	tasks	Number of ticks since SCH_Init() and the same time in ms (ticks × 10 ms)
SCH_Insert(idx, delay) (static)	SCH_Add_Task, SCH_Update	Inserts a slot into the delta list at the position given by its delay

3.2 Data Structure

Every task is described by the structure `sTasks` (Table 3.2). Compared with the structure of the handout (Program 1.7) the field `Next` was added to link the tasks. The tasks live in the static array `SCH_tasks_G[SCH_MAX_TASKS]` and are linked by index, not by pointer, so no dynamic memory is needed; the value `SCH_NO_TASK` (0xFF) is the "null" index. A structure takes 24 bytes on the Cortex-M3 (with padding), so the array of 10 tasks needs 240 bytes.

Table 3.2 — Fields of the structure sTasks

Field	Type	Meaning
pTask	void (*)(void)	Pointer to the task function; 0 means that the slot is free
Delay	uint32_t	Ticks to wait, relative to the previous node of the delta list
Period	uint32_t	Ticks between two runs; 0 for a one-shot task
RunMe	uint8_t	Number of times the task is due and has not been executed yet
TaskID	uint32_t	Index of the slot, i.e. the ID returned by SCH_Add_Task()
Next	uint8_t	Index of the next node of the delta list, or SCH_NO_TASK

Table 3.3 — Constants and global variables

Name	Value / type	Meaning
SCH_MAX_TASKS	10	Size of the task array
SCH_TICK_MS	10	Tick period in ms; it must be equal to the TIM2 period set in CubeMX
SCH_READY_SIZE	32	Size of the ready FIFO (31 entries can be used: one slot is left empty to tell full from empty)
SCH_NO_TASK	0xFF	"Null" index of the list
SCH_RETURN_NORMAL / SCH_RETURN_ERROR	0 / 1	Return codes of SCH_Delete_Task()

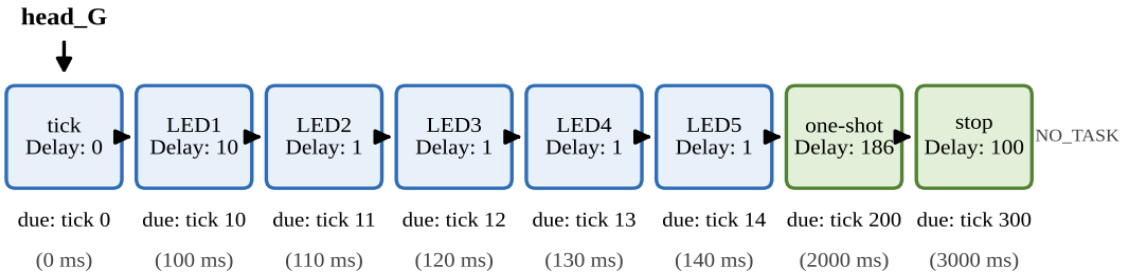
Name	Value / type	Meaning
Error_code_G	uint8_t	Last error: ERROR_SCH_TOO_MANY_TASKS (1), ERROR_SCH_CANNOT_DELETE_TASK (2), ERROR_SCH_READY_OVERFLOW (3)
tick_count_G	volatile uint32_t	Ticks since SCH_Init(); the time in ms is tick_count_G × SCH_TICK_MS

3.3 The Delta List

A simple scheduler decrements the Delay of every task in `SCH_Update()` (Program 1.9 of the handout). This is an O(n) operation at every tick and it is one of the designs that the handout calls unsatisfactory. Here the tasks are kept in a list sorted by due time and the Delay of each node is relative to the previous node, so the time of all the waiting tasks is counted by decrementing only the head of the list.

`SCH_Insert()` walks the list while the new delay is not smaller than the Delay of the current node and subtracts that Delay from the new one; the new node is inserted at this position and the Delay of the node behind it is reduced by the new Delay, so the absolute due time of the other tasks does not change. The walk continues while `delay >= Delay`, so a new task goes behind the tasks that are due at the same tick: tasks with the same due time are executed in the order in which they were inserted and the callbacks are delivered in the right order.

Figure 3.2 and Table 3.4 show the list right after the eight calls of `SCH_Add_Task()` of Section 6 (times in ticks of 10 ms). The 10 ms task, added with a delay of 0, is the head; LED1 is due 10 ticks after it, each following LED one tick later, then the one-shot task at tick 200 (Delay 186 after LED5) and the “stop” task at tick 300.



Absolute due time of a node = sum of the Delay fields from the head up to that node.

Figure 3.2: Delta list after the registration of the eight tasks of the demonstration

Table 3.4 — Content of the delta list after the registration

Position	Task	ID	Delay (Δ)	Due (tick)	Period (ticks)
1	task_print_tick	6	0	0	1
2	task_led1	0	10	10	50
3	task_led2	1	1	11	100
4	task_led3	2	1	12	150
5	task_led4	3	1	13	200

Position	Task	ID	Delay (Δ)	Due (tick)	Period (ticks)
6	task_led5	4	1	14	250
7	task_oneshot	5	186	200	0
8	task_stop_tick	7	100	300	0

3.4 Operation of the Scheduler

SCH_Update() is called at every tick:

1. the tick counter `tick_count_G` is incremented (it is the time base of `SCH_Get_Time_Ms()`);
2. if the Delay of the head is greater than 0 it is decremented;
3. while the head has Delay = 0, it is unlinked, its index is written in the ready FIFO and `RunMe` is incremented (if the FIFO is full, `Error_code_G = ERROR_SCH_READY_OVERFLOW`); if the task is periodic it is immediately inserted again with Delay = Period.

Re-inserting a periodic task in the interrupt, and not when the task has finished, means that its period does not drift even if the dispatcher is late.

SCH_Dispatch_Tasks() runs in the super loop:

1. while the FIFO is not empty, the index at `ready_tail` is taken;
2. with the interrupts masked, if the slot still holds a task and `RunMe > 0`, `RunMe` is decremented and the function pointer is copied (a task that was deleted after it became due is skipped);
3. the task is called outside the critical section, so the tick interrupt can still be served while it runs;
4. a one-shot task (`Period == 0`) frees its slot after it has run, so the slot can be reused.

SCH_Delete_Task() returns `SCH_RETURN_ERROR` and sets `ERROR_SCH_CANNOT_DELETE_TASK` if the ID is out of range or the slot is empty. Otherwise it unlinks the node and adds its Delay to the next node, so the other tasks keep their due time, and it clears the slot.

`SCH_Add_Task()`, `SCH_Delete_Task()` and the dispatcher modify data that is also used by the interrupt, so they run with the interrupts masked. The macros `ENTER_CRITICAL()` and `EXIT_CRITICAL()` save and restore the PRIMASK register, so they are also correct when they are called with the interrupts already disabled. A task may call `SCH_Add_Task()` or `SCH_Delete_Task()`; `task_stop_tick` does so.

4. STM32 Project Configuration

A project named lab4 was created in STM32CubeIDE for the STM32F103C6 with the configuration of Table 4.1. Every LED pin received a user label so that CubeMX generates the macros `<LABEL>_Pin` and `<LABEL>_GPIO_Port` in `main.h`.

Table 4.1 — CubeMX configuration of the project

Item	Setting
Clock	HSI 8 MHz, PLL disabled (SYSCLK = HCLK = PCLK1 = PCLK2 = 8 MHz), Flash latency 0
TIM2	Clock source: internal clock; counter mode: up; Prescaler = 7999; Counter Period = 9; auto-reload preload disabled; TIM2 global interrupt enabled in the NVIC
Tick period	8 MHz / (7999 + 1) / (9 + 1) = 100 Hz → 10 ms, equal to SCH_TICK_MS
USART1	Asynchronous, 115200 baud, 8 data bits, no parity, 1 stop bit, no hardware flow control, oversampling 16; PA9 = TX, PA10 = RX
PA5 … PA8, PB0	GPIO_Output, push-pull, no pull, low speed, initial level high (LED off); labels LED_1 … LED_4 and LED_OS

Listing 4.1 — TIM2 configuration generated by CubeMX (main.c)

```
htim2.Instance = TIM2;
htim2.Init.Prescaler = 7999;
htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
htim2.Init.Period = 9;
htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
  Error_Handler();
}
```

Listing 4.2 — USART1 configuration generated by CubeMX (main.c)

```
huart1.Instance = USART1;
huart1.Init.BaudRate = 115200;
huart1.Init.WordLength = UART_WORDLENGTH_8B;
huart1.Init.StopBits = UART_STOPBITS_1;
huart1.Init.Parity = UART_PARITY_NONE;
huart1.Init.Mode = UART_MODE_TX_RX;
huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart1.Init.OverSampling = UART_OVERSAMPLING_16;
if (HAL_UART_Init(&huart1) != HAL_OK)
{
  Error_Handler();
}
```

Listing 4.3 — GPIO configuration generated by CubeMX (main.c)

```
__HAL_RCC_GPIOA_CLK_ENABLE();
__HAL_RCC_GPIOB_CLK_ENABLE();


/*Configure GPIO pin Output Level */
HAL_GPIO_WritePin(GPIOA, LED_1_Pin|LED_2_Pin|LED_3_Pin|LED_4_Pin, GPIO_PIN_SET);


/*Configure GPIO pin Output Level */
HAL_GPIO_WritePin(LED_OS_GPIO_Port, LED_OS_Pin, GPIO_PIN_SET);


/*Configure GPIO pins : LED_1_Pin LED_2_Pin LED_3_Pin LED_4_Pin */
GPIO_InitStruct.Pin = LED_1_Pin|LED_2_Pin|LED_3_Pin|LED_4_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
```

```
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pin : LED_OS_Pin */
  GPIO_InitStruct.Pin = LED_OS_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(LED_OS_GPIO_Port, &GPIO_InitStruct);
```

5. Scheduler Implementation

The scheduler is a separate module: `scheduler.h` in `Core/Inc` and `scheduler.c` in `Core/Src`.

5.1 scheduler.h

The header declares the task structure, the constants, the error and return codes and the public functions. Only `SCH_Update()` and `SCH_Dispatch_Tasks()` are needed by the main program besides `SCH_Add_Task()`, `SCH_Delete_Task()` and `SCH_Init()`; the two getters give the tasks access to the time.

Listing 5.1 — Header of the scheduler (scheduler.h)

```
#ifndef INC_SCHEDULER_H_
#define INC_SCHEDULER_H_
#include "stdint.h"

typedef struct
{
    void (*pTask)(void);
    uint32_t Delay;
    uint32_t Period;
    uint8_t RunMe;
    uint32_t TaskID;
    uint8_t Next;
} sTasks;

#define SCH_MAX_TASKS 10 // temporarily we define max tasks = 10;
#define SCH_TICK_MS 10 // timer tick period in 10ms
#define SCH_READY_SIZE 32 // size of the ready FIFO
#define SCH_NO_TASK 0xFF// "null" index

// return code of the delete task SCH_Delete_Task()
#define SCH_RETURN_NORMAL 0
#define SCH_RETURN_ERROR 1

// error codes stored in Error_code_G
#define ERROR_SCH_TOO_MANY_TASKS 1
#define ERROR_SCH_CANNOT_DELETE_TASK 2
#define ERROR_SCH_READY_OVERFLOW 3

extern uint8_t Error_code_G;

void SCH_Init(void);

uint32_t SCH_Add_Task(void (*pFunction)(),
        uint32_t DELAY,
        uint32_t PERIOD); //this function serves as set timer

void SCH_Update(void); //this function serves as timer run, it will be called in ISR (every 10ms)
```

```
  void SCH_Dispatch_Tasks(void); //this function serves as flag checker, it will be called in
while(1){}

  uint8_t SCH_Delete_Task(uint32_t taskID);

  uint32_t SCH_Get_Tick(void);

  uint32_t SCH_Get_Time_Ms(void);

  #endif /* INC_SCHEDULER_H_ */
```

5.2 scheduler.c

Listing 5.2 contains the private data of the module and `SCH_Insert()`, the only function that knows how the delta list is built. It must be called with the interrupts disabled or from the interrupt itself.

Listing 5.2 — Data, critical-section macros and SCH_Insert() (scheduler.c)

```
  #include "scheduler.h"
  #include "main.h"

  static sTasks SCH_tasks_G[SCH_MAX_TASKS];
  static uint8_t head_G = SCH_NO_TASK;

  static volatile uint8_t ready_Q[SCH_READY_SIZE]; // FIFO of due task slots
  static volatile uint8_t ready_head = 0; // written by SCH_Update
  static volatile uint8_t ready_tail = 0; // read by Dispatch

  static volatile uint32_t tick_count_G = 0;
  uint8_t Error_code_G = 0;

  // protect the task list against the ISR while main modifies
  #define ENTER_CRITICAL()  uint32_t primask_ = __get_PRIMASK(); __disable_irq()
  #define EXIT_CRITICAL()   __set_PRIMASK(primask_)

  /* Insert task 'idx' so that it becomes due 'delay' ticks from now.
  The list is sorted by due time; each Delay is relative to the previous
  task. Equal due times keep insertion order (stable) -> correct order.
  Must be called with interrupts disabled (or from the ISR).*/

  static void SCH_Insert(uint8_t idx, uint32_t delay)
  {
      uint8_t prev = SCH_NO_TASK;
      uint8_t cur  = head_G;

      while (cur != SCH_NO_TASK && delay >= SCH_tasks_G[cur].Delay)
      {
          delay -= SCH_tasks_G[cur].Delay;
          prev = cur;
          cur  = SCH_tasks_G[cur].Next;
      }

      SCH_tasks_G[idx].Delay = delay;
      SCH_tasks_G[idx].Next  = cur;
      if (cur != SCH_NO_TASK)
      {
          SCH_tasks_G[cur].Delay -= delay;
      }

      if (prev == SCH_NO_TASK)
          head_G = idx;
      else
          SCH_tasks_G[prev].Next = idx;
  }
```

`SCH_Init()` puts every slot in the "free" state, empties the list and the FIFO and resets the tick counter and the error code. `SCH_Add_Task()` looks for the first free slot; if there is none, it sets `ERROR_SCH_TOO_MANY_TASKS` and returns `SCH_MAX_TASKS`. Otherwise it fills the slot, inserts it in the list with the requested delay and returns the index of the slot as the ID of the task.

Listing 5.3 — SCH_Init() and SCH_Add_Task() (scheduler.c)

```
void SCH_Init(void)
{
    for (uint8_t i = 0; i < SCH_MAX_TASKS; i++)
    {
        SCH_tasks_G[i].pTask  = 0;
        SCH_tasks_G[i].Delay  = 0;
        SCH_tasks_G[i].Period = 0;
        SCH_tasks_G[i].RunMe  = 0;
        SCH_tasks_G[i].TaskID = i;
        SCH_tasks_G[i].Next   = SCH_NO_TASK;
    }
    head_G       = SCH_NO_TASK;
    ready_head   = 0;
    ready_tail   = 0;
    tick_count_G = 0;
    Error_code_G = 0;
}

uint32_t SCH_Add_Task(void (*pFunction)(), uint32_t DELAY, uint32_t PERIOD)
{
    uint8_t idx;
    ENTER_CRITICAL();

    if (pFunction == 0)
    {
        EXIT_CRITICAL();
        return SCH_MAX_TASKS;
    }

    /* find a free slot */
    for (idx = 0; idx < SCH_MAX_TASKS; idx++)
    {
        if (SCH_tasks_G[idx].pTask == 0) break;
    }

    if (idx == SCH_MAX_TASKS)
    {
        Error_code_G = ERROR_SCH_TOO_MANY_TASKS;
        EXIT_CRITICAL();
        return SCH_MAX_TASKS;
    }

    SCH_tasks_G[idx].pTask  = pFunction;
    SCH_tasks_G[idx].Period = PERIOD;
    SCH_tasks_G[idx].RunMe  = 0;
    SCH_tasks_G[idx].TaskID = idx;
    SCH_Insert(idx, DELAY);

    EXIT_CRITICAL();
    return idx;
}
```

`SCH_Update()` is the function called in the timer interrupt. The loop at the end handles several tasks that become due in the same tick; because the delay of a task that follows

another one with the same due time is 0, they leave the list one after the other and are written to the FIFO in this order.

Listing 5.4 — SCH_Update() (scheduler.c)

```
/* Called every tick from the timer ISR.
Only the head of the list is decremented (O(1)). Tasks that reach 0 are
moved to the ready FIFO; periodic tasks are re-inserted immediately so
their period does not drift even if the dispatcher is late.*/
void SCH_Update(void)
{
    tick_count_G++;

    if (head_G == SCH_NO_TASK) return;

    if (SCH_tasks_G[head_G].Delay > 0)
        SCH_tasks_G[head_G].Delay--;

    while (head_G != SCH_NO_TASK && SCH_tasks_G[head_G].Delay == 0)
    {
        uint8_t idx = head_G;
        head_G = SCH_tasks_G[idx].Next;
        SCH_tasks_G[idx].Next = SCH_NO_TASK;

        uint8_t next_w = (uint8_t)((ready_head + 1) % SCH_READY_SIZE);
        if (next_w == ready_tail)
        {
            Error_code_G = ERROR_SCH_READY_OVERFLOW;   /* FIFO full */
        }
        else
        {
            ready_Q[ready_head] = idx;
            ready_head = next_w;
            SCH_tasks_G[idx].RunMe++;
        }

        if (SCH_tasks_G[idx].Period > 0)
        {
            SCH_Insert(idx, SCH_tasks_G[idx].Period);   /* periodic */
        }
    }
}
```

The dispatcher reads the FIFO written by the interrupt. The task is called outside the critical section; afterwards the slot of a one-shot task is released, but only if it still holds the same function and has not become due again.

Listing 5.5 — SCH_Dispatch_Tasks() (scheduler.c)

```
/* Called in the super loop: run all due tasks in FIFO order */
void SCH_Dispatch_Tasks(void)
{
    while (ready_tail != ready_head)
    {
        uint8_t idx = ready_Q[ready_tail];
        ready_tail = (uint8_t)((ready_tail + 1) % SCH_READY_SIZE);

        void (*task)(void) = 0;
        {
            ENTER_CRITICAL();
            if (SCH_tasks_G[idx].pTask != 0 && SCH_tasks_G[idx].RunMe > 0)
            {
                SCH_tasks_G[idx].RunMe--;
                task = SCH_tasks_G[idx].pTask;
            }
            EXIT_CRITICAL();
        }
```

```
        if (task != 0)
        {
            (*task)();   /* run the task to completion */

            /* one-shot task: free its slot after it has run */
            ENTER_CRITICAL();
            if (SCH_tasks_G[idx].pTask == task &&
                SCH_tasks_G[idx].Period == 0 &&
                SCH_tasks_G[idx].RunMe == 0)
            {
                SCH_tasks_G[idx].pTask = 0;
            }
            EXIT_CRITICAL();
        }
    }
}
```

When a task is deleted, its Delay is added to the next node so that the following tasks keep their due time. The two getters give the number of ticks and the time in ms since `SCH_Init()`.

Listing 5.6 — SCH_Delete_Task() and the time functions (scheduler.c)

```
uint8_t SCH_Delete_Task(uint32_t taskID)
{
    uint8_t ret;
    ENTER_CRITICAL();

    if (taskID >= SCH_MAX_TASKS || SCH_tasks_G[taskID].pTask == 0)
    {
        Error_code_G = ERROR_SCH_CANNOT_DELETE_TASK;
        ret = SCH_RETURN_ERROR;
    }
    else
    {
        /* unlink from the delta list (if it is still in it) */
        uint8_t prev = SCH_NO_TASK;
        uint8_t cur  = head_G;
        while (cur != SCH_NO_TASK && cur != (uint8_t)taskID)
        {
            prev = cur;
            cur  = SCH_tasks_G[cur].Next;
        }
        if (cur == (uint8_t)taskID)
        {
            uint8_t nxt = SCH_tasks_G[cur].Next;
            if (nxt != SCH_NO_TASK)
                SCH_tasks_G[nxt].Delay += SCH_tasks_G[cur].Delay;
            if (prev == SCH_NO_TASK)
                head_G = nxt;
            else
                SCH_tasks_G[prev].Next = nxt;
        }

        SCH_tasks_G[taskID].pTask  = 0;
        SCH_tasks_G[taskID].Delay  = 0;
        SCH_tasks_G[taskID].Period = 0;
        SCH_tasks_G[taskID].RunMe  = 0;
        SCH_tasks_G[taskID].Next   = SCH_NO_TASK;
        ret = SCH_RETURN_NORMAL;
    }

    EXIT_CRITICAL();
    return ret;
}

uint32_t SCH_Get_Tick(void)
{
    return tick_count_G;
}
```

```
uint32_t SCH_Get_Time_Ms(void)
{
    return tick_count_G * SCH_TICK_MS;
}
```

6. Application: Tasks and Main Program

6.1 Tasks

The application is in `main.c`. The function `print_log()` builds the string `[time ms] message` with `snprintf()`, using `SCH_Get_Time_Ms()` for the time, and sends it on USART1 with `HAL_UART_Transmit()`. Five periodic tasks toggle one LED each with `HAL_GPIO_TogglePin()` and print a message; `task_oneshot()` prints a message once; `task_print_tick()` prints the word “tick”; `task_stop_tick()` deletes the 10 ms task with `SCH_Delete_Task(tick_task_id)`. The macro `DEMO_TICK_PRINT` (set to 1) enables the two demonstration tasks that work with the 10 ms tick.

Listing 6.1 — Tasks and print_log() (main.c)

```
/* Print "[time ms] message" to the Virtual Terminal (USART1) */
static void print_log(const char *msg)
{
    char buf[64];
    int n = snprintf(buf, sizeof(buf), "[%lu ms] %s\r\n",
            (unsigned long)SCH_Get_Time_Ms(), msg);
    if (n > 0)
        HAL_UART_Transmit(&huart1, (uint8_t *)buf, (uint16_t)n, 100);
}

/* ---- 4 periodic tasks (LEDs are active-low: pin 0 = LED on) ---- */
void task_led1(void)   /* every 0.5 s */
{
    HAL_GPIO_TogglePin(LED_1_GPIO_Port, LED_1_Pin);
    print_log("LED1 toggled (period 500 ms)");
}

void task_led2(void)   /* every 1 s */
{
    HAL_GPIO_TogglePin(LED_2_GPIO_Port, LED_2_Pin);
    print_log("LED2 toggled (period 1000 ms)");
}

void task_led3(void)   /* every 1.5 s */
{
    HAL_GPIO_TogglePin(LED_3_GPIO_Port, LED_3_Pin);
    print_log("LED3 toggled (period 1500 ms)");
}

void task_led4(void)   /* every 2 s */
{
    HAL_GPIO_TogglePin(LED_4_GPIO_Port, LED_4_Pin);
    print_log("LED4 toggled (period 2000 ms)");
}

/* ---- 5th periodic task: every 2.5 s (uses the LED_OS pin) ---- */
void task_led5(void)
{
    HAL_GPIO_TogglePin(LED_OS_GPIO_Port, LED_OS_Pin);
    print_log("LED5 toggled (period 2500 ms)");
}

/* ---- one-shot task: runs once, no LED needed ---- */
```

```
void task_oneshot(void)
{
    print_log("ONE-SHOT task executed");
}

#if DEMO_TICK_PRINT
/* ---- demo: callback every 10 ms printing the timestamp ---- */
void task_print_tick(void)
{
    print_log("tick");
}

/* ---- one-shot at 3 s: delete the 10 ms task (demo SCH_Delete_Task) ---- */
void task_stop_tick(void)
{
    if (SCH_Delete_Task(tick_task_id) == SCH_RETURN_NORMAL)
        print_log("10 ms tick task DELETED");
}
#endif
```

Table 6.1 summarises the tasks registered in `main()`. Delays and periods are written in ticks (1 tick = 10 ms).

Table 6.1 — Tasks registered with SCH_Add_Task()

Task	Delay (ticks)	Period (ticks)	Real period	Purpose
task_led1	10	50	0.5 s	Toggle LED_1 (PA5)
task_led2	11	100	1 s	Toggle LED_2 (PA6)
task_led3	12	150	1.5 s	Toggle LED_3 (PA7)
task_led4	13	200	2 s	Toggle LED_4 (PA8)
task_led5	14	250	2.5 s	Toggle LED_5 (PB0, label LED_OS)
task_oneshot	200	0	once	Print a message once, after 2 s
task_print_tick	0	1	10 ms	Print the timestamp at every tick
task_stop_tick	300	0	once	After 3 s delete the 10 ms task (SCH_Delete_Task)

All five periods are multiples of 500 ms and the initial delays of the LED tasks differ by one tick, so no two LED tasks are ever due in the same tick (the advice about task overlap in Section 2.3.11 of the handout). The 10 ms task is due at every tick, so it shares some ticks with the other tasks; the other task is then called first, because it was inserted in the list before the 10 ms task was inserted again.

6.2 Initialisation and Main Loop

In `main()` the scheduler is initialised, the timer interrupt is started, a start message is printed and all the tasks are registered before the super loop is entered. The identifier of the 10 ms task is saved in `tick_task_id` for the deletion. The super loop contains only the dispatcher.

Listing 6.2 — Initialisation and registration of the tasks (main.c)

```
  SCH_Init();
  HAL_TIM_Base_Start_IT(&htim2);          /* start the 10 ms tick */

  print_log("SYSTEM START");
```

```
/* Delays/periods are in ticks (1 tick = 10 ms) */
SCH_Add_Task(task_led1, 10, 50);
SCH_Add_Task(task_led2, 11, 100);
SCH_Add_Task(task_led3, 12, 150);
SCH_Add_Task(task_led4, 13, 200);
SCH_Add_Task(task_led5, 14, 250);   /* 2.5 s */
SCH_Add_Task(task_oneshot, 200, 0);  /* once, after 2 s */


#if DEMO_TICK_PRINT
tick_task_id = SCH_Add_Task(task_print_tick, 0, 1);
SCH_Add_Task(task_stop_tick, 300, 0);
#endif
```

Listing 6.3 — Super loop (main.c)

```
while (1)
{
  /* USER CODE END WHILE */


  /* USER CODE BEGIN 3 */
    SCH_Dispatch_Tasks();


}
```

The only code executed in the interrupt is the call of `SCH_Update()` from the HAL callback of the timer.

Listing 6.4 — Timer interrupt callback (main.c)

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim2)
{
    if(htim2->Instance == TIM2)
        SCH_Update();
}
```

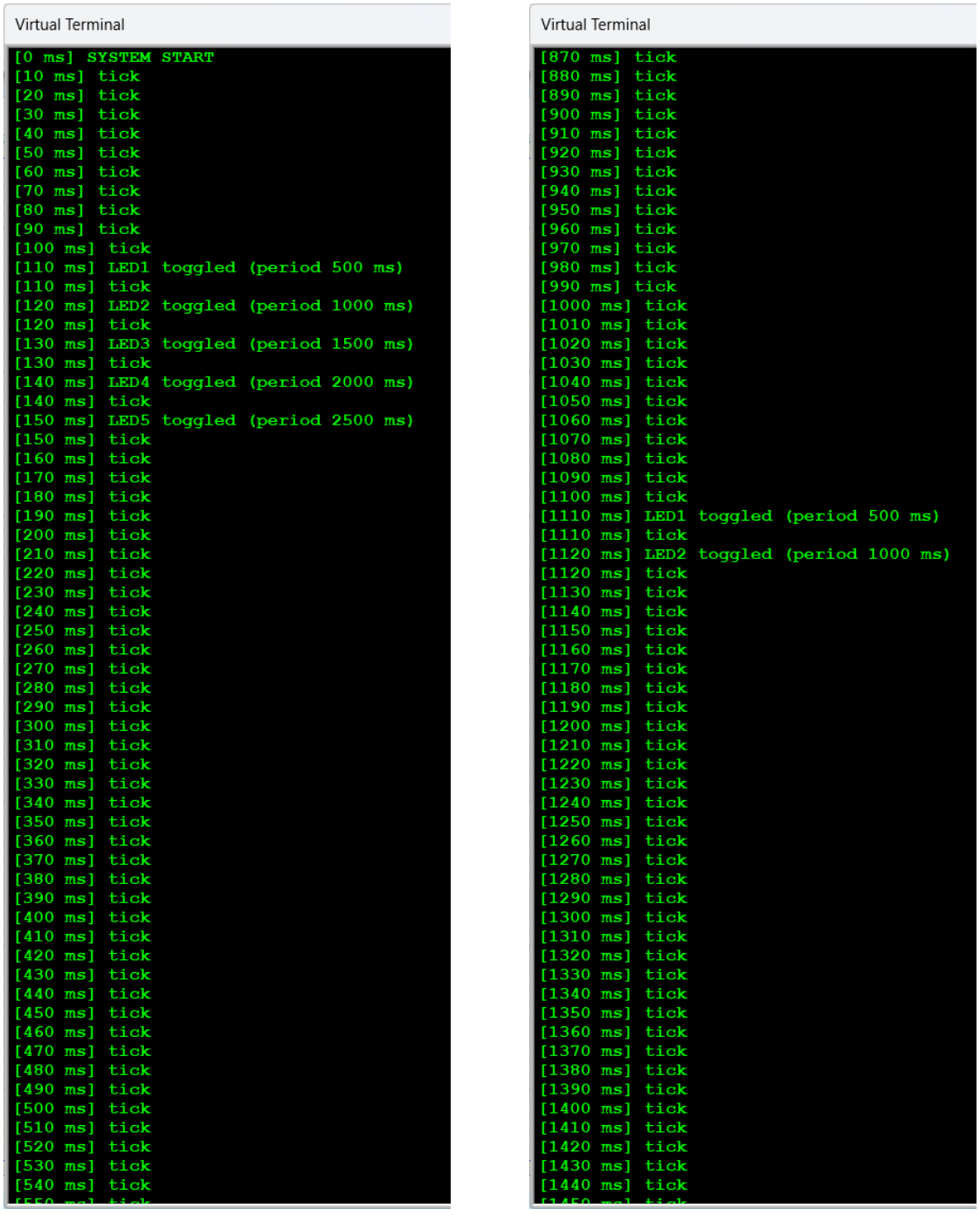
7. Simulation Results

7.1 Test Setup

The project was built in STM32CubeIDE and the program was run on the STM32F103C6 of the Proteus schematic of Figure 2.1. The Virtual Terminal uses 115200 baud, 8 data bits, no parity and 1 stop bit. Every line has the form `[t ms] message`, where t is the value of `SCH_Get_Time_Ms()` at the moment in which the task runs; it is always a multiple of 10 ms.

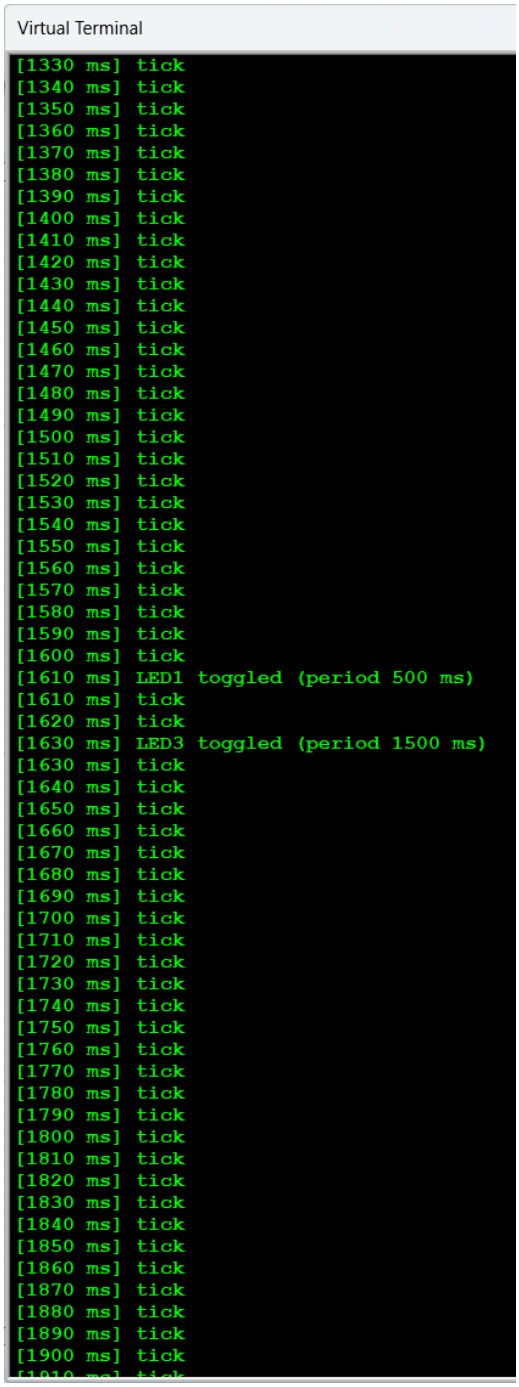
7.2 Virtual Terminal Output

Figures 7.1 to 7.3 show the output of the Virtual Terminal. The five captures overlap each other, except between 540 ms and 870 ms.

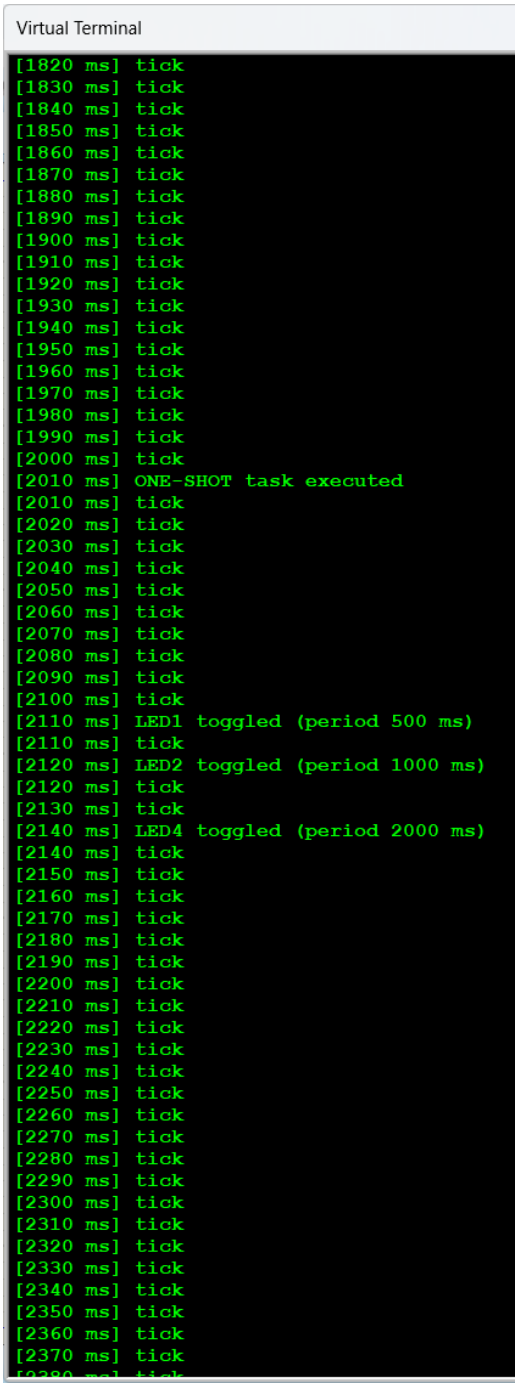


(a) 0 – 540 ms *(b) 870 – 1440 ms*

Figure 7.1: Virtual Terminal output from the start of the program

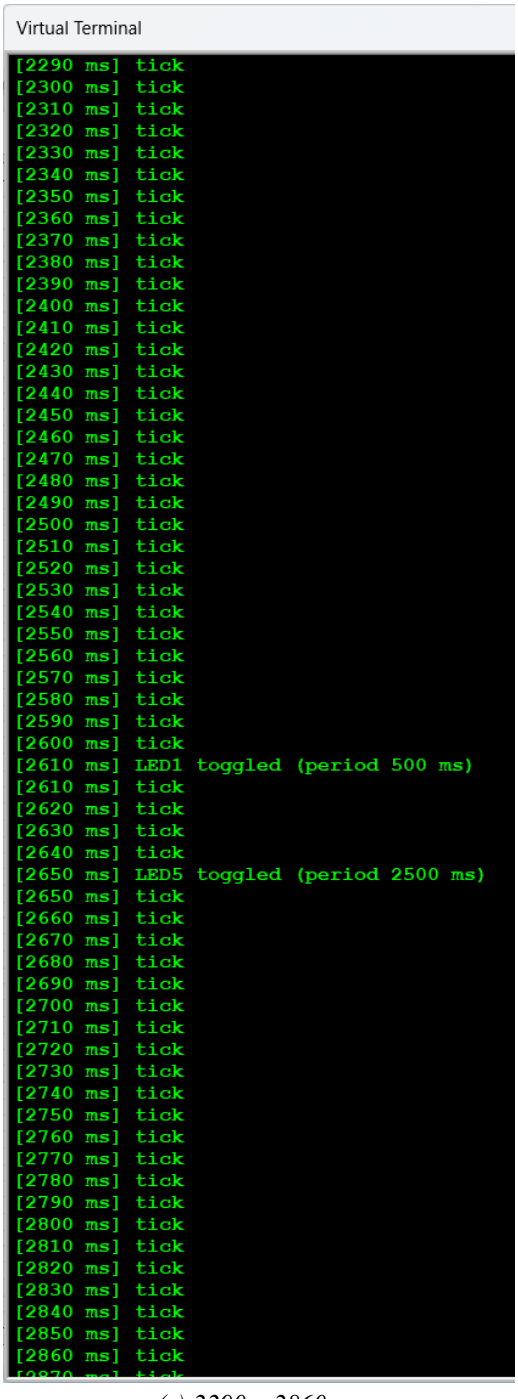


(c) 1330 – 1900 ms



(d) 1820 – 2370 ms

Figure 7.2: Virtual Terminal output around 1.6 s and 2 s



(e) 2290 – 2860 ms

Figure 7.3: Virtual Terminal output around 2.6 s

The following points can be read from the captures:

- **Start and 10 ms tick.** The program prints `[0 ms] SYSTEM START` and then the word “tick” every 10 ms from 10 ms on. In all the captures the timestamps increase by exactly 10 ms, so the timer tick is regular and the callback registered every 10 ms is called at every tick (Figures 7.1 – 7.3).
- **Several timeouts at the same time.** The five LED tasks start at 110, 120, 130, 140 and 150 ms (Figure 7.1 a), one tick after each other, as their initial delays are 10 to 14

ticks. In every tick that contains a task, the line of the task is printed before the "tick" line, i.e. the callbacks are delivered in the order of insertion.

- **Periodic tasks.** LED1 runs again at 1110 ms, 1610 ms, 2110 ms and 2610 ms; LED2 at 1120 ms and 2120 ms; LED3 at 1630 ms; LED4 at 2140 ms; LED5 at 2650 ms (Figures 7.1 b – 7.3). The line of LED1 expected at 610 ms is in the gap between the captures (a) and (b).
- **One-shot task.** `ONE-SHOT task executed` appears once, at 2010 ms (Figure 7.2 d), and does not appear again in the remaining part of the captures.

Table 7.1 — Execution times read from the Virtual Terminal

Task	Period	Times in the captures (ms)	Differences (ms)
task_led1	500 ms	110, 1110, 1610, 2110, 2610	1000 (= 2 × 500, 610 ms is in the gap), 500, 500, 500
task_led2	1 s	120, 1120, 2120	1000, 1000
task_led3	1.5 s	130, 1630	1500
task_led4	2 s	140, 2140	2000
task_led5	2.5 s	150, 2650	2500
task_oneshot	once	2010	—

7.3 Analysis of the Timestamps

All the differences of Table 7.1 are equal to the programmed periods, so the periods are exact and do not drift: a periodic task is inserted again in the list inside the interrupt, with a delay equal to its period.

The first execution of every task is one tick (10 ms) later than the value "delay × 10 ms" (Table 7.2). The reason is the 10 ms task: it is registered with a delay of 0, so it is the head of the list with Delay = 0. `SCH_Update()` decrements the head only when its Delay is greater than 0, so the first tick is used to release this task and the count-down of the other tasks starts one tick later. The offset is the same for all the tasks, it does not accumulate and the relative timing of the tasks is not changed. It is smaller than the resolution of 10 ms of the timestamps that is required by the handout.

Table 7.2 — First execution of the tasks

Task	Delay (ticks)	Delay × 10 ms	First run observed (ms)	Difference
task_led1	10	100 ms	110	+10 ms
task_led2	11	110 ms	120	+10 ms
task_led3	12	120 ms	130	+10 ms
task_led4	13	130 ms	140	+10 ms
task_led5	14	140 ms	150	+10 ms
task_oneshot	200	2000 ms	2010	+10 ms

Figure 7.4 shows the sequence of the executions up to 3.2 s. It was obtained by running the file `scheduler.c` tick by tick on a PC with the same tasks and delays; the points that are visible in Figures 7.1 – 7.3 coincide with the terminal output.

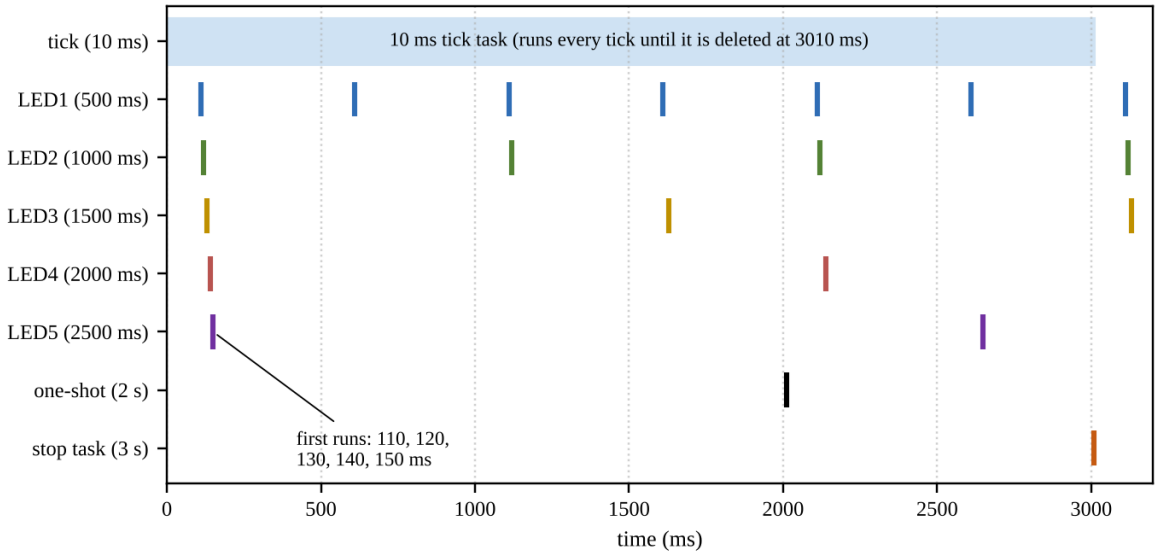


Figure 7.4: Timeline of the executions of the tasks up to 3.2 s

7.4 Deleting a Task

`task_stop_tick` is registered with a delay of 300 ticks and, with the offset of Section 7.3, it runs at 3010 ms. It calls `SCH_Delete_Task(tick_task_id)` with the ID that `SCH_Add_Task(task_print_tick, 0, 1)` returned and, when the function returns `SCH_RETURN_NORMAL`, it prints `10 ms tick task DELETED`. The captures of Figures 7.1 – 7.3 end at 2860 ms, so this moment is not in them. The expected output from 3 s on, following from the code (and from Figure 7.4), is:

```
[3000 ms] tick
[3010 ms] 10 ms tick task DELETED
[3110 ms] LED1 toggled (period 500 ms)
[3120 ms] LED2 toggled (period 1000 ms)
[3130 ms] LED3 toggled (period 1500 ms)
```

The “tick” lines stop after 3000 ms, while the periodic tasks go on with the same times as before, because the Delay of the deleted node was added to the next node of the list.

8. Compliance with the Requirements

Table 8.1 — Requirements of the handout and how they are met

Requirement	How it is met
Four functions: SCH_Update, SCH_Dispatch_Tasks, SCH_Add_Task (returns an ID), SCH_Delete_Task (by ID)	Implemented in scheduler.c with the prototypes of the handout (Listings 5.3 – 5.6)
Five periodic tasks of 0.5, 1, 1.5, 2 and 2.5 s	task_led1 … task_led5 with periods of 50, 100, 150, 200 and 250 ticks (Table 6.1); periods confirmed in Table 7.1

Requirement	How it is met
Regular 10 ms timer tick	TIM2 with Prescaler 7999 and Period 9 (Table 4.1); "tick" lines every 10 ms (Figures 7.1 – 7.3)
A callback every 10 ms that prints the time	task_print_tick (delay 0, period 1) prints `[t ms] tick` with t from SCH_Get_Time_Ms()
Timestamps accurate to the nearest 10 ms	t = tick counter × 10 ms; the tick counter is incremented in the timer interrupt
More than one timeout registered at a time	Eight tasks are registered at the same time (up to SCH_MAX_TASKS = 10)
Calls to SCH_Add_Task() before the main loop	All tasks are registered before `while (1)`; their callbacks print the timestamp when they run
Not a single timeout only	The delta list holds up to 10 tasks
Callbacks in the right order	Stable insertion in the delta list and FIFO dispatching: tasks that are due in the same tick run in the order of insertion (e.g. [110 ms] LED1 before [110 ms] tick)
No O(n) search in SCH_Update()	Only the head of the list is counted down. The list is walked only when a task expires (to insert a periodic task again), at most SCH_MAX_TASKS = 10 nodes
Interrupt frequency	The handout asks for a regular 10 ms tick (100 Hz). The interrupt only calls SCH_Update(): it does not print and it does not wait
Deleting a task	task_stop_tick deletes the 10 ms task with SCH_Delete_Task() (Section 7.4)
Errors	Error_code_G is set when the task array is full, when a task cannot be deleted or when the ready FIFO overflows

9. Conclusion

A cooperative scheduler was designed and implemented on the STM32F103C6. One timer interrupt of 10 ms drives any number of tasks (up to SCH_MAX_TASKS); the tasks are kept in a delta list, so the interrupt counts down only one value per tick, equal-time tasks are called in the right order, and periodic tasks do not drift. The simulation in Proteus showed five periodic tasks with exact periods of 0.5 s to 2.5 s, a one-shot task, a 10 ms task that prints the timestamp and the deletion of a task, all in one program with a super loop that only calls `SCH_Dispatch_Tasks()`.

The following points are the limits of the present design:

- The scheduler is cooperative: a task is not interrupted, so it must be short compared with the tick. In this demonstration the longest line (41 characters) needs about 3.6 ms on the UART at 115200 baud and the two lines of a tick need about 5 ms, less than 10 ms. A longer task delays the following tasks; the ticks are still counted and the due tasks wait in the FIFO (31 entries), and `ERROR_SCH_READY_OVERFLOW` is set if it is full.
- `HAL_UART_Transmit()` is a blocking call. It is acceptable for the demonstration, but in a real application the transmission could be done with interrupts or DMA.

- The time printed is the time at which the task runs, not the time at which it became due; the two are equal when the CPU is free.
- The task added with a delay of 0 causes the constant offset of one tick described in Section 7.3. It could be avoided by counting the tick also on the first node whose Delay is greater than 0.
- The optional parts of the handout (idle mode in `SCH_Go_To_Sleep()`, watchdog, `SCH_Report_Status()`) were not implemented; errors are only stored in `Error_code_G`.

10. Demo Video and Links

Demo video:
https://drive.google.com/file/d/16fMvfH7QNpokHGv8QrZbbOEcRKYXEt00/view?usp=sharing

Source code: garrettnhan-coder/mcu-mpu_lab_schematic at lab4_source_code

Lab report: